

AI-Powered Animated Video Generation System

Agentic AI Semester Project 2026

Group 6

i221326 Abdul Momin

i220961 Hamza Naveed

i221042 Maaz Ali

i221053 Huzaifa Nasir

May 5, 2026

Abstract

This report presents an end-to-end, agentic pipeline that converts a single natural-language prompt into a short animated film. The system is implemented as a five-phase workflow: story and script generation, audio synthesis and mixing, video composition, a full-stack web interface for orchestration, and an edit agent with undo and version history. Each phase reads and updates a shared JSON state, enabling re-runs and targeted edits without reprocessing the entire pipeline. The implementation integrates LLM-based planning, TTS, image generation, FFmpeg-based compositing, and a LangGraph-driven edit agent with tool execution and state snapshots.

Contents

1	Introduction	3
2	System Overview	3
2.1	Architecture Diagram	3
3	Shared JSON Schema	3
3.1	Schema Excerpt	4
4	Phase 1: Story and Script Generation	4
5	Phase 2: Audio Generation and Integration	4
5.1	Audio Timing Strategy	5
6	Phase 3: Video Generation and Composition	5
6.1	Visual Prompting Strategy	5
7	Phase 4: Web Interface and Orchestration	5
7.1	API Surface	5
8	Phase 5: Edit Agent with Undo	6
8.1	Edit Flow Diagram	6
8.2	Intent Classification	6
8.3	Tool Execution and Selective Reruns	6
8.4	State Versioning and Undo	7
8.5	Versioning Sequence	7
9	Orchestration and Execution	7
10	Testing	8
10.1	Test Coverage Summary	8
11	Results	8
11.1	Visual Artifacts	9
12	Challenges and Mitigations	9
13	Limitations and Future Work	9
14	Individual Contributions	9
15	Conclusion	10
16	Deliverables Checklist	10
A	Mermaid Sources	10
A.1	Architecture Diagram	10
A.2	Edit Agent Flow	10
A.3	Data Flow	10
A.4	Versioning Sequence	10

1 Introduction

Creating animated video content traditionally requires multiple specialists and long production timelines. This project demonstrates how a multi-agent workflow can automate that pipeline for short animated films. Our system accepts a single prompt and outputs a composited MP4 while exposing phase-level control, progress updates, and edit functionality. The project is structured to be modular and testable, with a shared schema that connects all phases.

2 System Overview

The pipeline is implemented as five modules, each with clear responsibilities and inputs/outputs.

User Prompt

|

v

Phase 1: Story + Script + Characters (LLM)

|

v

Phase 2: Dialogue TTS + BGM + Timing (Audio)

|

v

Phase 3: Scene Images + Animation + Sync (Video)

|

v

Phase 4: Web UI + Orchestration (FastAPI + React)

|

v

Phase 5: Edit Agent + Undo (LangGraph + State Manager)

Each phase can run independently using CLI scripts or through the web application, and every run updates a central JSON state object stored under `data/outputs/`.

2.1 Architecture Diagram

Figure 1 summarizes the main components. The diagram image can be generated from the Mermaid source in Appendix A.

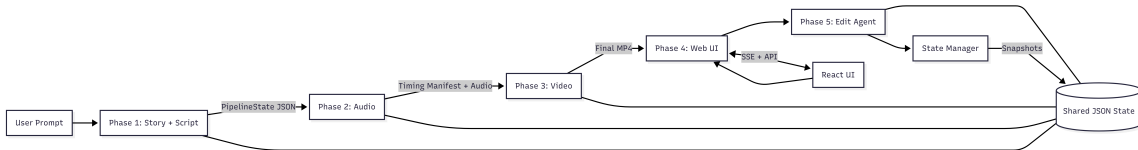


Figure 1: End-to-end pipeline architecture and data flow.

3 Shared JSON Schema

A single JSON schema defines the contract between phases. It includes:

- **meta**: project id, prompt, timestamps, schema version, duration, tags.
- **story**: logline, synopsis, genre, tone, and target duration.
- **characters**: id, name, role, voice profile, and visual profile.
- **scenes**: ordered scenes with setting, mood, visual prompt, and dialogue lines.

- **audio:** per-line audio references, BGM tracks, and a timing manifest.
- **video:** scene assets, final video path, subtitle information.
- **edits:** version history and current version id.
- **phases:** per-phase status and error metadata.

The schema is enforced via Pydantic models, with JSON Schema validation to ensure phase outputs are consistent. The schema enables selective re-runs and deterministic updates.

3.1 Schema Excerpt

The schema is implemented in `shared/schemas/pipeline_state.schema.json`. Table 1 highlights the top-level fields and their roles.

Field	Purpose
meta	Project identifiers, timestamps, schema version, and total duration.
story	Logline, synopsis, tone, and target duration.
characters	Character roster with voice and visual profiles.
scenes	Ordered scenes with visual prompts and dialogue.
audio	Per-line audio assets, BGM tracks, and timing manifest.
video	Scene assets, subtitle data, and final MP4 path.
edits	Version history and snapshot metadata.
phases	Status map for phase progress and errors.

Table 1: PipelineState top-level schema fields.

4 Phase 1: Story and Script Generation

Phase 1 expands the user prompt into a structured narrative and scene plan. It uses a Groq-hosted LLM (model: `llama-3.3-70b-versatile`) with strict JSON formatting rules. The planner ensures:

- 3–6 scenes totaling 15–45 seconds.
- 2–4 characters with consistent gender and visual descriptions.
- Scene dialogue lines linked to character ids.
- BGM moods normalized to a controlled vocabulary.

The output is validated and normalized before it becomes the canonical `PipelineState`. A repair pass is included to fix schema deviations from the LLM when necessary.

5 Phase 2: Audio Generation and Integration

Phase 2 converts dialogue into audio and assembles background music (BGM) for each scene. It performs:

- TTS synthesis per dialogue line using ElevenLabs or an offline fallback.
- Timing manifest generation with start and end timestamps.
- Scene-level BGM selection by mood from a local library.

- Mixing of dialogue and BGM with configurable gain controls.

The phase writes per-line WAV files, per-scene mixes, and a final concatenated audio track. Outputs are stored under `data/outputs/phase2/<project_id>/`.

5.1 Audio Timing Strategy

We compute a dialogue timeline by summing line durations and a configurable inter-line gap. The timing manifest aligns each dialogue line to a global timeline and becomes the source of truth for video synchronization. BGM is trimmed or padded to match scene duration so the final mixed audio remains consistent with scene timing.

6 Phase 3: Video Generation and Composition

Phase 3 renders scene visuals and synchronizes them with audio. The implementation includes:

- Image generation with Pollinations, conditioned by scene prompts and character visuals.
- Per-speaker image prompts to align lip-sync and dialogue segments.
- Ken Burns style animation and concatenation using FFmpeg.
- Optional Wav2Lip integration for per-segment lip-sync.
- Subtitle generation and burn-in as an optional step.

Scene assets, clips, and final MP4 outputs are stored in `data/outputs/phase3/<project_id>/` with metadata logs for prompt and lip-sync decisions.

6.1 Visual Prompting Strategy

We generate per-speaker prompts to ensure that each dialogue segment has a matching character image, which improves lip-sync quality and keeps character identity consistent. The prompts combine scene tone, camera framing, and character visual profiles.

7 Phase 4: Web Interface and Orchestration

The web interface provides a user-facing control plane for the pipeline. The backend is built with FastAPI and exposes:

- Per-phase run endpoints and a full-pipeline endpoint.
- SSE streaming for live progress updates.
- State and video retrieval endpoints.
- Edit agent endpoints for classification and undo operations.

The frontend is built with React and Vite. It includes status cards, live event logs, a video preview panel, and edit and undo interfaces. The UI supports phase-level reruns and refreshes state and history from the backend.

7.1 API Surface

Table 2 lists the core API routes implemented by the backend.

Endpoint	Purpose
GET /api/projects	List stored project ids, state paths, and video paths.
GET /api/status/{project_id}	Phase status, errors, and latest artifacts.
GET /api/stream/{project_id}	Server-sent events for live progress updates.
POST /api/run/phase1	Run Phase 1 with a prompt.
POST /api/run/phase2	Run Phase 2 for an existing project.
POST /api/run/phase3	Run Phase 3 for an existing project.
POST /api/run/full	Execute all phases sequentially.
POST /api/edit	Submit a natural-language edit request.
POST /api/edit/undo	Revert to a previous snapshot version.

Table 2: Core API endpoints.

8 Phase 5: Edit Agent with Undo

Phase 5 allows natural-language edits after the initial render. The edit agent uses LangGraph with tool bindings and a state manager for snapshots.

8.1 Edit Flow Diagram

Figure 2 captures the classification and tool-execution loop. The diagram image can be generated from the Mermaid source in Appendix A.

8.2 Intent Classification

An LLM-based classifier parses edit requests into a structured object:

```
{
  "intent": "change_voice_tone",
  "target": "audio",
  "scope": "character:char_1",
  "parameters": {"tone": "whispered"}
}
```

The classifier supports targets such as `audio`, `video_frame`, `video`, and `script`. When ambiguity exists, the agent queries project context before taking action.

8.3 Tool Execution and Selective Reruns

The edit agent routes requests to tool functions that update the JSON state and delete targeted cached assets to force regeneration. Examples include:

- Update scene background and delete affected images and clips.
- Update character visuals and remove dependent assets.
- Update voice tone and rerun audio generation.
- Update BGM mood and remix the video.

After state updates, the agent can re-run the appropriate pipeline phase on request.

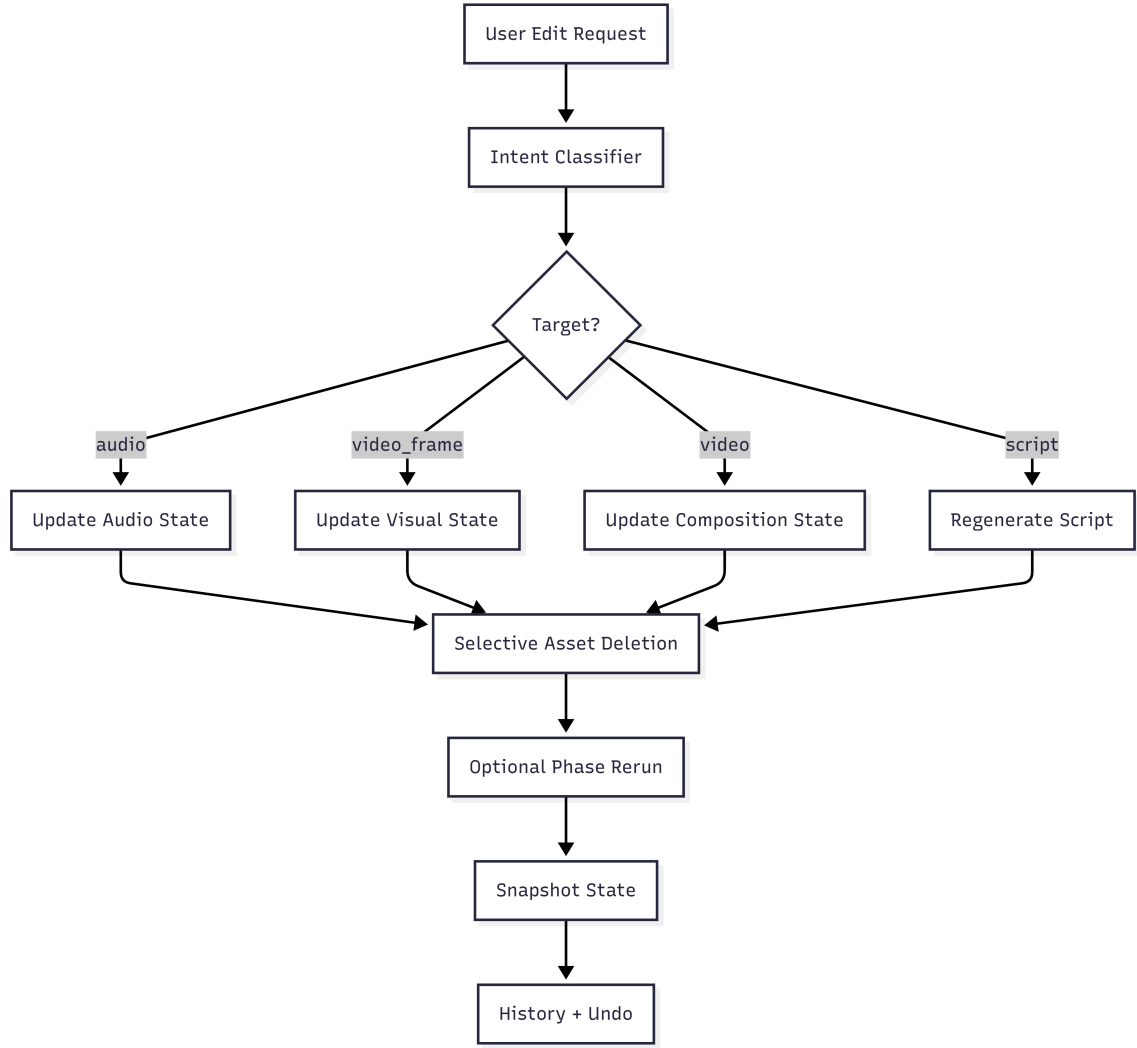


Figure 2: Edit agent classification and tool-execution loop.

8.4 State Versioning and Undo

The system maintains an append-only history of snapshots for each project. A snapshot captures the state JSON and all referenced assets. The UI exposes a history panel to revert to any prior version, restoring the exact pipeline state and files.

8.5 Versioning Sequence

Figure 3 shows how snapshots are taken during edits and how a revert restores a previous state.

9 Orchestration and Execution

Phase execution can be triggered via CLI scripts or via the web API. The backend maintains an in-memory job store and broadcasts progress over SSE. Each phase updates the state and writes a phase-complete record. The LangGraph edit graph runs with a SQLite checkpointer to preserve conversational context and tool call history.

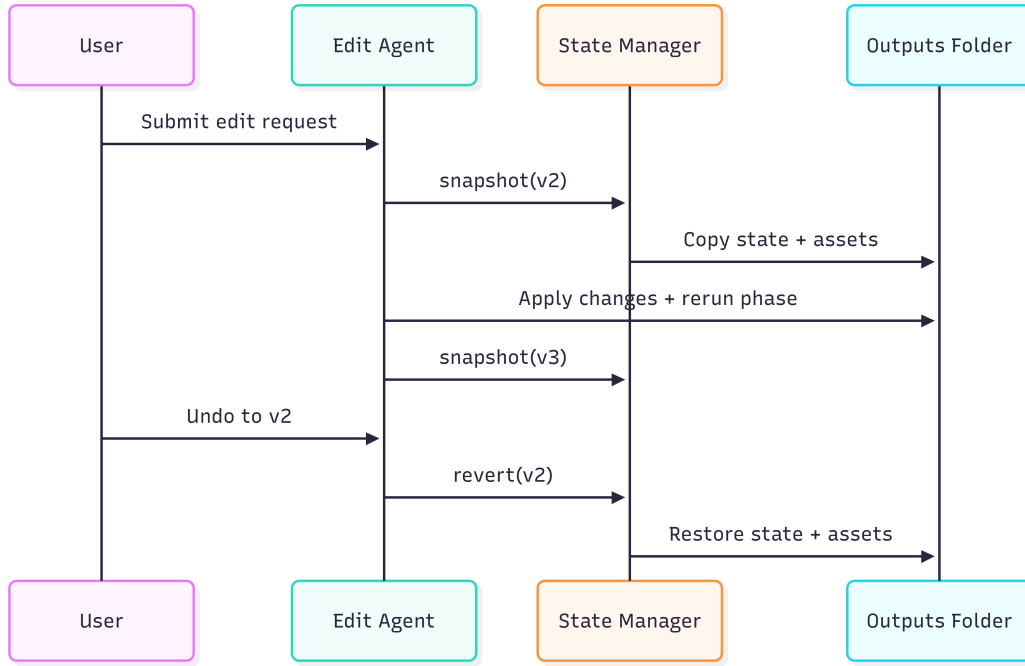


Figure 3: Snapshot and undo sequence.

10 Testing

Unit tests are provided for multiple agents, including story, audio, and video components, as well as edit intent classification. Tests verify schema validity, tool routing behavior, and pipeline state updates. Test files are located under `agents/*/tests` and `tests/`.

10.1 Test Coverage Summary

Key test areas include:

- Story agent JSON schema validation and normalization.
- Audio agent timing manifest and asset generation.
- Video agent asset creation and output paths.
- Edit intent classification for multiple query types.
- State manager snapshot and revert behavior.

11 Results

We verified the pipeline using the sample project `proj_601cadb0`. The system produced:

- A structured Phase 1 state JSON with validated characters and scenes.
- Full audio output with timing manifest and scene mixes.
- A composited MP4 video with optional subtitle burn-in.
- Successful edit requests with state snapshots and undo operations.

The outputs are stored in `data/outputs/phase1/`, `data/outputs/phase2/`, and `data/outputs/phase3/`. Phase-level reruns were verified through both CLI and the web interface.

11.1 Visual Artifacts

Figure 4 provides a data flow view and where artifacts are produced in the repository. The diagram can be generated from Appendix A.

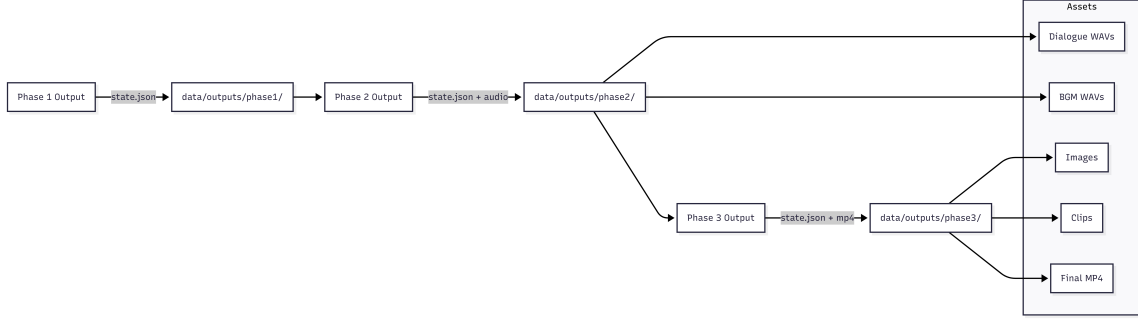


Figure 4: Artifact flow across phases and outputs.

12 Challenges and Mitigations

- **LLM output drift:** Implemented schema validation with repair prompts to enforce JSON structure.
- **Dialogue alignment:** Enforced speaker-turn segmentation to keep timing consistent.
- **Voice and visual consistency:** Normalized gender and aligned visual descriptors to voices.
- **BGM balance:** Exposed tunable gain controls and debug logging.
- **Edit safety:** Required context lookups and used targeted asset deletion to avoid full reruns.

13 Limitations and Future Work

- Add richer visual variety per scene with multiple images and transitions.
- Improve lip-sync robustness and face detection fallbacks.
- Expand edit intents to include pacing, camera motion, and subtitle styling.
- Provide a Docker-based installer for easier deployment.
- Add richer evaluation metrics for story coherence and A/V sync quality.

14 Individual Contributions

All members collaborated across phases with shared ownership of the JSON schema and cross-phase integration. Each member contributed to design, implementation, testing, and documentation for their respective modules:

- Abdul Momin
- Hamza Naveed
- Maaz Ali
- Huzaiifa Nasir

15 Conclusion

This project demonstrates a complete, modular agentic pipeline for prompt-to-video generation. The shared state contract enables reliable handoff between phases, while the edit agent and undo system provide practical post-generation control. The system fulfills the requirements for a full-stack AI pipeline with multi-agent orchestration and serves as a foundation for further research and productization.

16 Deliverables Checklist

Table 3 maps the required artifacts to their locations in the repository or submission package.

Artifact	Location / Notes
GitHub repository	Public repo <code>AgenticAI_Project_Group_6</code> (submit link).
Project report (PDF)	This report (exported from <code>Project_Report.tex</code>).
Demo video (3–7 min)	Screen recording showing prompt-to-video, 3 edits, 2 undo.
Presentation slides	PDF or PPTX, 10–15 minutes demo flow.
Sample outputs	<code>data/outputs/phase1, phase2, phase3</code> .
Unit tests	<code>agents/*/tests</code> and <code>tests/</code> .

Table 3: Submission artifacts required by the assignment.

A Mermaid Sources

This appendix includes the Mermaid source files used to generate figures. Render them locally or in a Mermaid-compatible tool and export PNGs to the `diagrams/` folder.

A.1 Architecture Diagram

Mermaid source placeholder: `diagrams/architecture.mmd`

A.2 Edit Agent Flow

Mermaid source placeholder: `diagrams/edit_flow.mmd`

A.3 Data Flow

Mermaid source placeholder: `diagrams/data_flow.mmd`

A.4 Versioning Sequence

Mermaid source placeholder: `diagrams/versioning.mmd`

References

- Groq API and LLM models
- ElevenLabs TTS
- Pollinations image generation

- FFmpeg and MoviePy tooling
- LangChain and LangGraph
- FastAPI and React